Terraform Task-2

Task Description:

Create 2 EC2 instances on 2 different regions and install nginx using terraform script.

Step 1 : Configured aws cli in terraform server.

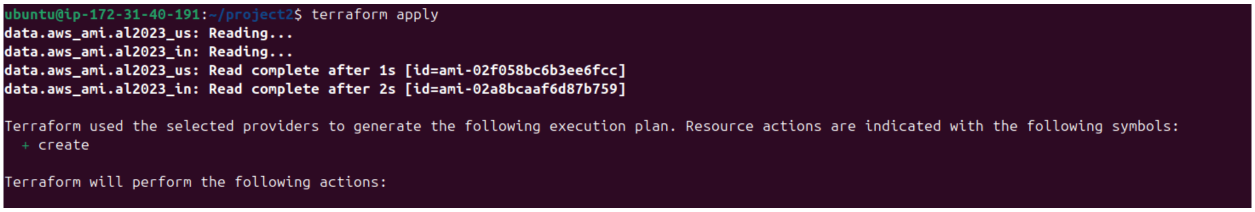
```
ubuntu@ip-172-31-40-191:~$ aws configure
AWS Access Key ID [None]: AKIA4F35FI6UL3HODB6O
AWS Secret Access Key [None]: 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
Default region name [None]: us-west-2
Default output format [None]:
```

Step 3 - Executed the script

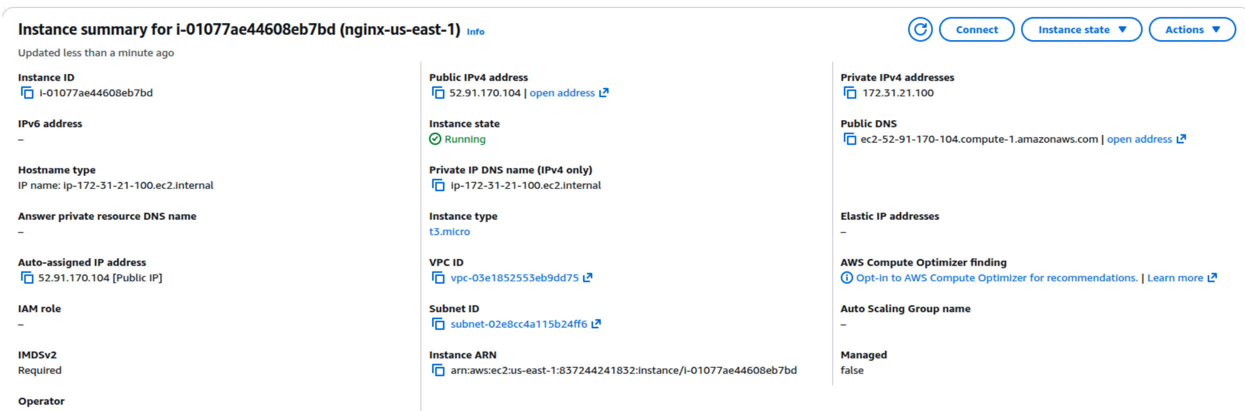
terraform init

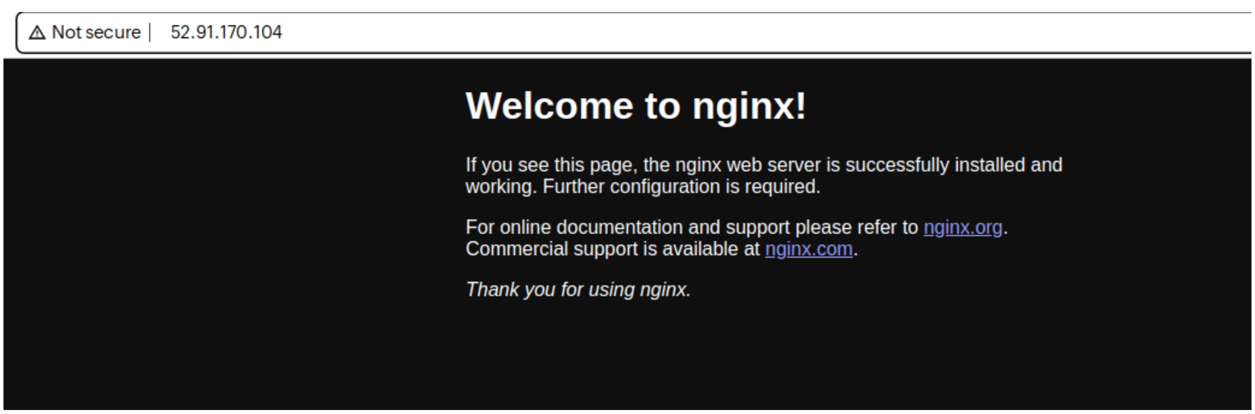
```
ubuntu@ip-172-31-40-191:~/project2$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

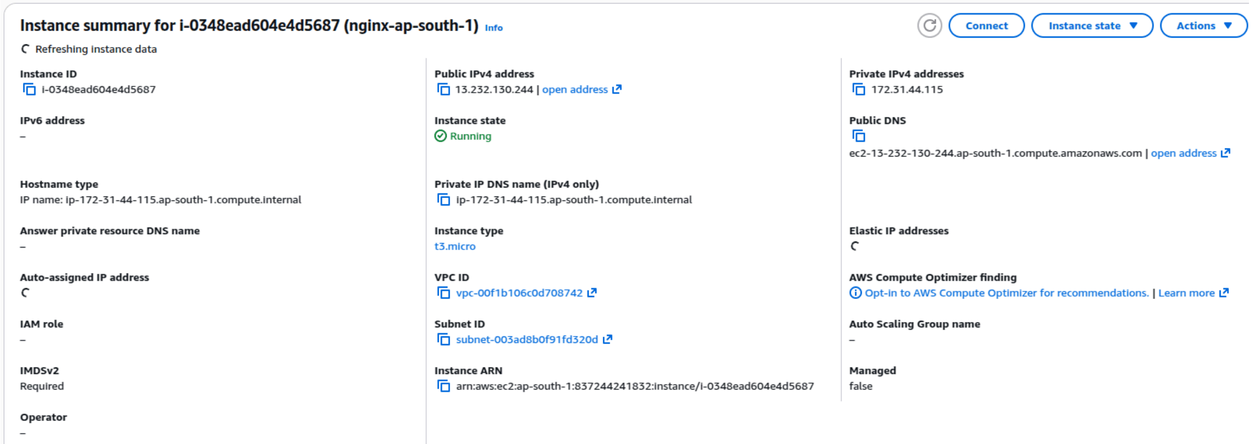
terraform apply

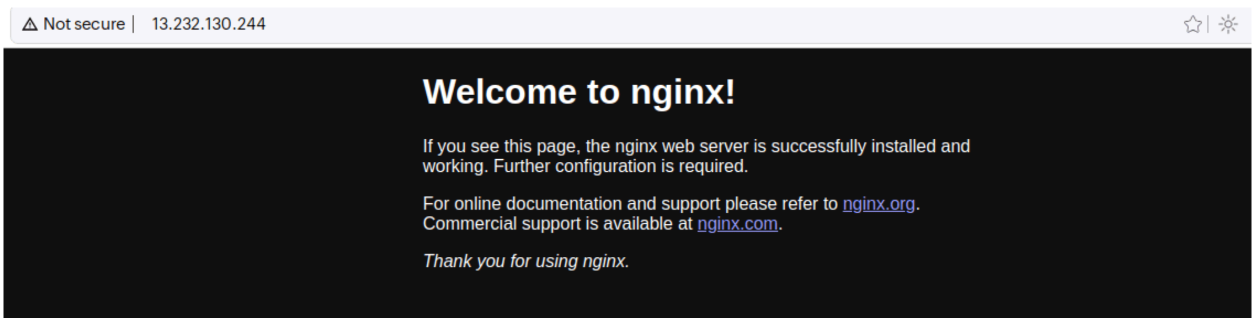


Instances started running in two regions









Step 2 - Created a terraform script to launch instances in N. Verginia and Mumbai

```
terraform {
 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = "~> 5.0"
  }
 }
}
provider "aws" {
```

```
  alias  = "us"

  region = "us-east-1"

}

provider "aws" {

  alias  = "in"

  region = "ap-south-1"

}

data "aws_ami" "al2023_us" {

  provider    = aws.us

  most_recent = true

  owners      = ["amazon"]

  filter {

    name   = "name"

    values = ["al2023-ami-*-x86_64"]

  }

}

data "aws_ami" "al2023_in" {

  provider    = aws.in

  most_recent = true

  owners      = ["amazon"]

  filter {

    name   = "name"

    values = ["al2023-ami-*-x86_64"]

  }

}

resource "aws_security_group" "web_us" {
```

```
  provider = aws.us
  name     = "nginx-sg-us"
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
resource "aws_security_group" "web_in" {
  provider = aws.in
  name     = "nginx-sg-in"
  ingress {
    from_port   = 22
```

```
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
resource "aws_instance" "nginx_us" {
  provider      = aws.us
  ami           = data.aws_ami.al2023_us.id
  instance_type = "t3.micro"
  security_groups = [
    aws_security_group.web_us.name
  ]
  user_data = <<-EOF
    #!/bin/bash
```

```
    dnf update -y
    dnf install -y nginx
    systemctl enable nginx
    systemctl start nginx
  EOF
  tags = {
    Name = "nginx-us-east-1"
  }
}
resource "aws_instance" "nginx_in" {
  provider      = aws.in
  ami           = data.aws_ami.al2023_in.id
  instance_type = "t3.micro"
  security_groups = [
    aws_security_group.web_in.name
  ]
  user_data = <<-EOF
    #!/bin/bash
    dnf update -y
    dnf install -y nginx
    systemctl enable nginx
    systemctl start nginx
  EOF
  tags = {
    Name = "nginx-ap-south-1"
  }
```

```
}
```